面向对象程序设计实践（C++）
课程实验报告

题目	电商交易平台设计与实现
姓名	马一民
学号	2023211196
班级	2023211311

目录

1 前言 0
1.1 题目概述 0
1.2 开发环境 0
1.3 功能需求 0
1.3.1 客户功能需求： 0
1.3.2 网络版需求 0
1.3.3 其他附加功能 1
2 总体设计 2
3 详细设计 3
3.1 子系统客户端详细设计 3
3.1.1 总体介绍 3
3.1.2 表现层（UIL） 3
3.1.2.1 功能介绍： 3
3.1.2.2 类之间的关系： 4
3.1.2.3 详细类介绍 4
3.1.3 业务逻辑层（BLL） 11
3.1.3.1 功能介绍 11
3.1.3.2 类之间的关系 11
3.1.3.3 详细类介绍 12
3.1.4 网络通信层（NL） 19
3.1.4.1 功能介绍 19
3.1.4.2 详细类介绍 19
3.1.5 各层通信的一个实例 21
3.2 子系统服务端详细设计 22
3.2.1 总体介绍 22
3.2.2 网络通信层（NL） 22
3.2.2.1 功能介绍 22
3.2.2.2 类之间的关系 23
3.2.2.3 详细类介绍 23
3.2.3 业务逻辑层（BLL） 26
3.2.3.1 功能介绍 26
3.2.3.2 类之间的关系 26
3.2.3.3 详细类介绍 26
3.2.4 数据访问层（DAL） 30
3.2.4.1 功能介绍 30
3.2.4.2 类之间的关系 31
3.2.4.3 详细类介绍 31
3.2.5 各层通信的一个实例 35
3.3 数据库说明 35
3.3.1 文件说明 35
3.3.2 关系数据库说明 36
3.3.2.1 商品数据库 36

3.3.2.2 订单数据库.....37
3.3.2.3 具体的订单商品数据库.....37
3.3.2.4 购物车数据库.....38
3.4 接口协议说明.....39
3.4.1 协议数据单元 (PDU) 的语法.....39
3.4.2 协议数据单元 (PDU) 的语义.....40
3.4.3 底层承载协议.....40
3.5 UI 说明.....41
3.5.1 客户端展示.....41
3.5.2 服务端展示.....43
4 实现.....44
4.1 单机版实现过程的问题：.....44
4.1.1 新环境的熟悉问题.....44
4.1.2 面向对象的整体设计问题.....44
4.1.3 其他具体问题.....45
4.2 网络版实现过程的问题：.....45
4.2.1 仍旧存在的面向对象的整体设计问题：.....45
4.3 经验教训：.....45

1 前言

1.1 题目概述

本综合题目的的任务是使用 C++ 语言，基于面向对象的程序设计方法，设计并实现一个简单的电商交易平台，提供用户管理、商品管理、交易管理等功能。并且要求为网络版，采用 CS 结构，客户端和服务器端为不同的进程。

1.2 开发环境

Qt Creator 16.0.2
MySQL Sever 8.0.42
MySQL Workbench 8.0.42

1.3 功能需求

1.3.1 客户功能需求：

1) 用户注册登录 ：支持新用户注册平台账号，已注册用户用平台账号登录平台，要求已注册用户的信息长久保留。
2) 修改账户密码： 支持登录后对用户账号的密码修改。
3) 余额管理： 支持用户账号中余额的查询、充值、消费等 。
4) 添加商品 ：支持商家添加新商品，要求已添加的商品信息长久保留。
5) 展示平台商品信息： 支持针对不同类型用户、无论登录与否均展示平台商品信息。
6) 搜索平台商品信息 ：支持依据某种条件（比如：名称）对平台商品进行筛选并展示筛选结果 。
7) 商品信息管理 ：支持商家对其商品的信息进行管理，包括但不限于价格管理、剩余量管理等。
8) 统一折扣：支持对同一品类下所有产品打折的活动。
9) 购物车管理：支持消费者向购物车添加、删除指定数量的商品，也支持消费者修改当前购物车商品的拟购买数量。
10) 订单生产：选择购物车的商品生成订单，计算并显示订单总金额。
11) 网上支付：消费者使用余额支付订单，支付成功后，消费者被扣除的余额应转至商家余额中。

1.3.2 网络版需求

1) 要求采用 C/S 结构，客户端与服务器系统之间使用 socket 进行通信。

2) 支持一定的错误场景处理能力。

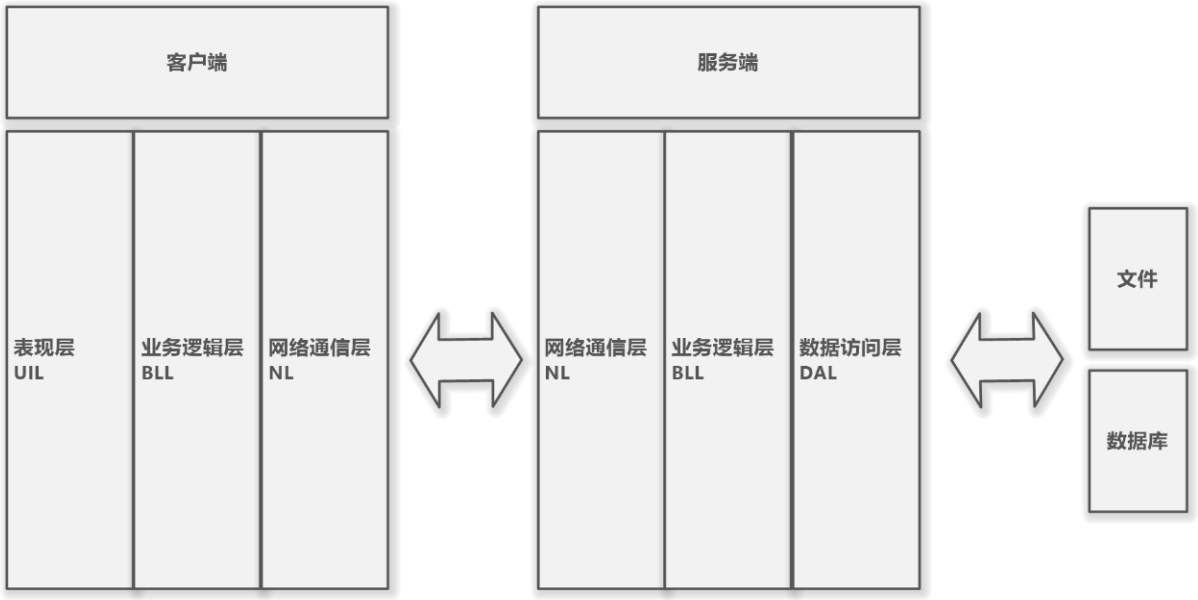
1.3.3 其他附加功能

1) 本实现使用了 Qt，设计了较为美观的 GUI。
2) 使用 MySQL 数据库存储信息。

2 总体设计

本系统可以划分为客户端和服务端两个子系统，两个子系统之间通过 socket 建立 TCP 连接进行通信，中间传送的数据包采用 json 格式。当客户登录客户端时，首先尝试与服务端建立连接，若建立连接成功，后续的操作将可以与服务端进行通信。服务端接收客户端操作传来的信号，完成对数据库和文件的修改。

其中客户端分为表现层（UIL），业务逻辑层（BLL），网络通信层（NL）三层。服务端分为网络通信层（NL），业务逻辑层（BLL），数据访问层（DAL）三层。这里各层之间是通过对象的接口调用和 Qt 独特的信号槽机制进行互相通信联系。详细介绍见下文详细设计。

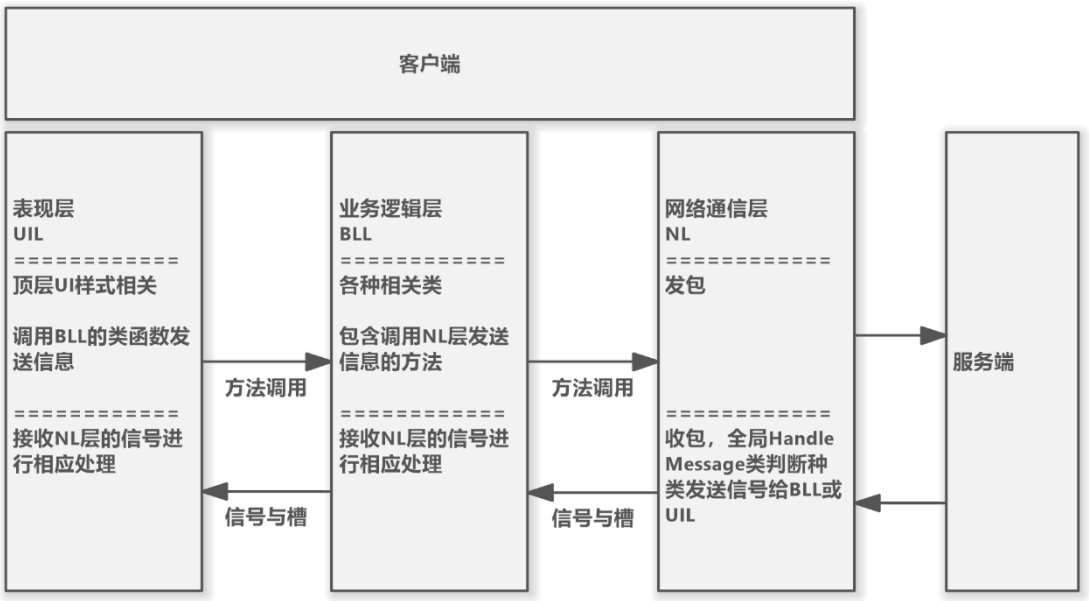


3 详细设计

3.1 子系统客户端详细设计

3.1.1 总体介绍

客户端分为表现层（UIL），业务逻辑层（BLL），网络通信层（NL）三层。各层之间通过对象的接口调用和 Qt 独特的信号槽机制进行互相通信联系。

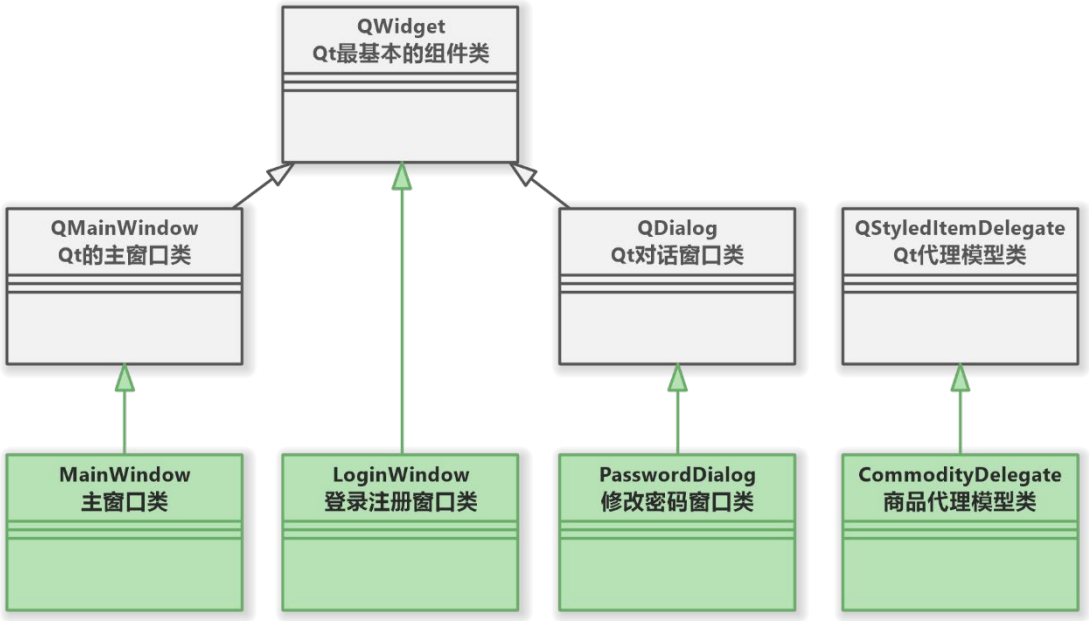


3.1.2 表现层（UIL）

3.1.2.1 功能介绍：

表现层即为展示给用户的 UI，这里主要完成了各 UI 组件的展示，UI 界面的布局，以及用户对 UI 进行操作之后的响应。

3.1.2.2 类之间的关系：



这里箭头表示继承，白色即为 Qt 自带的类，绿色为我设计的类，继承自 Qt 原有的类，其中继承自 QWidget 的三个窗口类会以窗口的形式展示出来。也就是说整个 UI 包括主窗口，登陆注册窗口和修改密码窗口三个窗口。剩下的商品代理模型类是用于从数据库读取到的信息建立代理模型，并以更美观多样的方式展示出来的类。

3.1.2.3 详细类介绍

下面给出各自类的设计：
maindow 类最复杂，因为所有的 UI 界面的每一个组件，每一次交互都需要在这里体现。
这里的 public 只有构造函数和析构函数。
private 成员变量一般是某一个组件的指针，也有 BLL 层类的对象指针。
private 中的方法一般是指仅用于本窗口内的方法，不涉及信号的连接。
private slots 则是指槽函数，涉及信号的连接，既包括接收其他层的信号作出处理的函数，也包括本层用户交互需要的处理函数。
所有的方法和成员变量均已加上注释。

```
/**
 * @brief 主窗口类，电商平台的主界面控制器
 */
class MainWindow : public QMainWindow
{
    Q_OBJECT

public:
    MainWindow(QWidget *parent = nullptr); // 构造函数
```

```
    ~MainWindow(); // 析构函数

private slots:
    // 基础交互槽函数
    void onCommoditySelected(const QModelIndex &index); // 主页商品选择
    void onLoginButtonClicked(); // 登录按钮点击
    void onLogoutButtonClicked(); // 登出按钮点击
    void onLoginWindowClosed(const QString& username,const QString& password,const
QString& type,unsigned long long balance); // 登录窗口关闭
    void onCommoditySelectedPage3(const QModelIndex &index); // 商品管理页商品选择
    void onBuyNowClicked(); // 立即购买按钮点击
    void onAddToCartClicked(); // 加入购物车按钮点击
    void updateTotalPrice(int quantity); // 更新总价显示

    // 购物车相关槽函数
    void updateCartDisplay(); // 更新购物车显示
    void onCartQuantityChanged(int id, int quantity); // 购物车数量改变
    void onRemoveCartItem(int id); // 移除购物车项目
    void onClearCartClicked(); // 清空购物车按钮点击
    void onCreateOrderClicked(); // 创建订单按钮点击

    // 订单相关槽函数
    void onOrderSelected(const QModelIndex &index); // 订单选择
    void onPayOrderClicked(); // 支付订单按钮点击
    void onDeleteOrderClicked(); // 删除订单按钮点击

    // 用户账户相关槽函数
    void onChangePasswordClicked(); // 修改密码按钮点击
    void onChargeButtonClicked(); // 充值按钮点击

    // 商品管理相关槽函数(商家专用)
    void onNewGoodsButtonClicked(); // 新增商品按钮点击
    void onDeleteGoodsButtonClicked(); // 删除商品按钮点击
    void onChangeGoodsButtonClicked(); // 修改商品按钮点击
    void onClearGoodsButtonClicked(); // 清空商品表格按钮点击
    void onUniDiscoutButtonClicked(); // 统一折扣设置按钮点击

    // 网络响应处理槽函数
    void handleFetchAllInfoOfCommodityResponse(bool success, const QString& message,
const QVariantMap& commodityData); // 主页商品信息获取响应
    void handleFetchAllInfoOfCommodityResponse_page3(bool success, const QString&
message, const QVariantMap& commodityData); // 商品管理页商品信息获取响应
    void handlePurchaseResponse(bool success, const QString& message, unsigned long long
newBuyerBalance, unsigned int newCommodityStock); // 购买响应
```

```
    void handleCommodityUpdateResponse(bool success, const QString& message); // 商品更新响应
    void handleCommodityNewResponse(bool success, const QString& message); // 新增商品响应
    void handleDeleteCommodityResponse(bool success, const QString& message, unsigned int deletedCommodityId); // 删除商品响应
    void handleSetUniDiscountResponse(bool success, const QString& message,float uniDiscount,const QString& sourceStoreName,const QString& type); // 设置统一折扣响应
    void handleChangePasswordResponse(bool success, const QString& message); // 修改密码响应
    void handleRechargeResponse(bool success, const QString& message, unsigned long long newBalance); // 充值响应

    // 购物车操作响应处理
    void onCartUpdated(bool success, const QString &message, const QList<CartItem> &items, const QJsonArray &cartItemsJson); // 购物车更新信号处理
    void handleUpdateCartItemQuantityResponse(bool success, const QString& message, const QString& action, const QJsonArray& updatedCartItems); // 更新购物车项目数量响应
    void handleAddToCartResponse(bool success, const QString& message, const QJsonArray& updatedCartItems); // 加入购物车响应
    void handleRemoveCartItemResponse(bool success, const QString& message, const QString& action, const QJsonArray& updatedCartItems); // 移除购物车项目响应
    void handleClearCartResponse(bool success, const QString& message, const QString& action, const QJsonArray& updatedCartItems); // 清空购物车响应

    // 订单操作响应处理
    void handleFetchOrdersResponse(bool success, const QString& message, const QJsonArray& orders); // 获取订单列表响应
    void handleDeleteOrderResponse(bool success, const QString& message, unsigned int deletedOrderId); // 删除订单响应
    void handleValidateOrderItemsResponse(const QString& status, const QString& message, const QJsonArray& validatedItems, unsigned long long totalPrice, const QString& changes); // 订单验证响应
    void handleCreateOrderResponse(bool success, const QString& message, unsigned long long newBalance, unsigned long long totalPrice); // 创建订单响应
    void handlePayOrderResponse(bool success, const QString& message, unsigned int orderId, unsigned long long paidAmount, unsigned long long newBalance); // 支付订单响应

private:
    Ui::MainWindow *ui; // UI 对象指针

    // 状态标志位
    bool isLogin=false; // 登录状态标志
```

```
    bool isMerchant=false; // 商家身份标志

    // 子窗口
    loginwindow* lwindow; // 登录窗口指针

    // 数据库相关
    QSqlTableModel *model; // 数据库表模型

    // 业务对象
    User* currentUser=nullptr; // 当前用户对象指针
    Commodity* currentCommodity=nullptr; // 当前选中商品对象指针

    // 主界面布局组件
    QListWidget* menuBar; // 左侧菜单栏
    QStackedWidget* pages; // 页面堆栈容器

    // 左侧个人信息区域组件
    QLabel* l_profile_welcome; // 欢迎标签
    QLabel* l_profile_userName; // 用户名标签
    QLabel* l_profile_balance; // 余额标签
    QPushButton* l_profile_login; // 登录按钮
    QPushButton* l_profile_logout; // 登出按钮
    QPushButton* l_profile_charge; // 充值按钮
    QPushButton* l_profile_changePassword; // 修改密码按钮

    // 左侧菜单项组件
    QListWidgetItem *item1; // 主页菜单项
    QListWidgetItem *item2; // 购物车菜单项
    QListWidgetItem *item3; // 商品管理菜单项
    QListWidgetItem *item4; // 其他功能菜单项
    QListWidgetItem *fakeitem2; // 购物车占位项(未登录)
    QListWidgetItem *fakeitem3; // 商品管理占位项(未登录)
    QListWidgetItem *fakeitem3_notMerchant; // 商品管理占位项(非商家)
    QListWidgetItem *fakeitem4; // 其他功能占位项(未登录)
    QListWidgetItem *item5; // 我的订单菜单项
    QListWidgetItem *fakeitem5; // 我的订单占位项(未登录)

    // 页面 1 - 主页商品浏览
    QLineEdit *filterEdit; // 搜索筛选输入框
    QComboBox *filterCombo; // 筛选条件下拉框
    QStandardItemModel *itemModel; // 商品列表数据模型
    QListView *listView; // 商品列表视图
    QTableWidget* r_commodity_table; // 右侧商品详情表格
    QLabel *unloggedLabel4; // 未登录提示标签
```

```
    QSpinBox *buyQuantitySpinBox; // 购买数量选择器
    QLabel *totalPriceLabel; // 总价格标签
    QPushButton *addToCartButton; // 加入购物车按钮
    QPushButton *buyNowButton; // 直接购买按钮

    // 页面 2 - 购物车
    QTableWidget *cartTable; // 购物车商品表格
    QPushButton *clearCartButton; // 清空购物车按钮
    QPushButton *createOrder; // 创建订单按钮
    QLabel *emptyCartLabel; // 购物车为空提示标签

    // 页面 3 - 商品管理(商家专用)
    QLineEdit *filterEdit_page3; // 商品管理页搜索框
    QComboBox *filterCombo_page3; // 商品管理页筛选框
    QListView *listView_page3; // 商品管理页商品列表
    QTableWidget *r_commodity_table_page3; // 商品管理页详情表格
    QLabel* originalPrice; // 原价标签
    QLabel* nowPrice; // 现价标签
    QPushButton* rsb_b_buy; // 购买按钮
    QPushButton* rsb_b_change; // 修改商品按钮
    QPushButton* rsb_b_new; // 新增商品按钮
    QPushButton* rsb_b_clear; // 清空表格按钮
    QPushButton* rsb_b_delete; // 删除商品按钮
    QLabel* tip; // 操作提示标签
    QLineEdit *uniDiscoutEdit; // 统一折扣输入框
    QComboBox *uniDiscoutCombo; // 统一折扣类型选择框
    QPushButton *uniDiscoutButton; // 统一折扣设置按钮

    // 页面 5 - 我的订单
    QListView *orderListView; // 订单列表视图
    QTableWidget *r_order_table; // 订单基本信息表格
    QTableWidget *r_order_items_table; // 订单商品详情表格
    QLabel *r_order_total_label; // 订单总价标签
    QPushButton *r_order_pay_button; // 支付订单按钮
    QPushButton* r_order_delete_button; // 删除订单按钮

    // 私有初始化方法
    void initWidget(); // 初始化组件
    void initLayout(); // 初始化布局
    void initMerchantTablePermissions(); // 初始化商家表格权限
    void clearMerchantTableInfo(); // 清空商家表格信息
    void updateOrderDisplay(); // 更新订单显示
    void updateOrderDisplayFromMemory(); // 从内存更新订单显示
```

```
    // 商品数据请求相关方法
    void refreshCommodityListViews(); // 刷新主页商品列表
    void refreshMerchantCommodityListViews(); // 刷新商家商品列表
    void requestCommodities(const QString& storeName, const QString& context); // 请求商品数据
    void handleCommoditiesResponse(const QJsonDocument& responseDoc, const QString& context); // 处理商品数据响应
    void populateModel(QStandardItemModel* model, const QList<QVariantMap>& commodities, bool isPage3); // 填充商品数据模型
};
```

其余的类与 mainwindow 类详细，不再过多介绍，下面给出另外两个窗口类的代码：loginwindow 类，所有的方法和成员变量均已加上注释。

```
/**
 * @brief 登录窗口类，处理用户登录和注册界面
 */
class loginwindow : public QWidget
{
    Q_OBJECT

public:
    loginwindow(QWidget *parent = nullptr); // 构造函数
    ~loginwindow(); // 析构函数
    void initloginUI(); // 初始化登录界面 UI

private slots:
    // 用户交互槽函数
    void onLoginButtonClicked(); // 登录按钮点击
    void onRegisterButtonClicked(); // 注册按钮点击
    void onToRegisterButtonClicked(); // 切换到注册界面按钮点击
    void onToLoginButtonClicked(); // 切换到登录界面按钮点击

    // 网络响应处理槽函数
    void onRegisterResponse(bool success, const QString& message); // 注册响应处理
    void onLoginResponse(bool success,const QString& message,const QString& username,const QString& password,const QString& type,unsigned long long balance); // 登录响应处理

private:
    // 输入组件
    QLineEdit *m_usernameEdit; // 用户名输入框
    QLineEdit *m_passwordEdit; // 密码输入框

    // 按钮组件
```

```
    QPushButton *loginButton; // 登录按钮
    QPushButton *registerButton; // 注册按钮
    QPushButton *toLoginButton; // 切换到登录按钮
    QPushButton *toRegisterButton; // 切换到注册按钮

    // 选择组件
    QCheckBox *iAmMerachant; // 商家身份选择框

signals:
    void loginwindowclosed(const QString& username,const QString& password,const
QString& type,unsigned long long balance); // 登录窗口关闭信号
};
```

PasswordDialog 类，所有的方法和成员变量均已加上注释。

```
/**
 * @brief 密码修改对话框类，处理用户密码修改界面
 */
class PasswordDialog : public QDialog
{
    Q_OBJECT
public:
    explicit PasswordDialog(QWidget *parent = nullptr); // 构造函数

    // 获取方法
    QString getOldPassword() const; // 获取旧密码
    QString getNewPassword() const; // 获取新密码

private slots:
    void onConfirmClicked(); // 确认按钮点击槽函数

private:
    // 输入组件
    QLineEdit *oldPasswordEdit; // 旧密码输入框
    QLineEdit *newPasswordEdit; // 新密码输入框
    QLineEdit *confirmPasswordEdit; // 确认密码输入框

    // 数据存储
    QString newPassword; // 验证通过的新密码
};
```

最后的 CommodityDelegate 类比较简单，该类仅仅是完成根据传入模型，“绘制”新的模型展示界面。所有的方法和成员变量均已加上注释。

```
/**
 * @brief 商品卡片委托类，自定义商品列表项的绘制样式
```

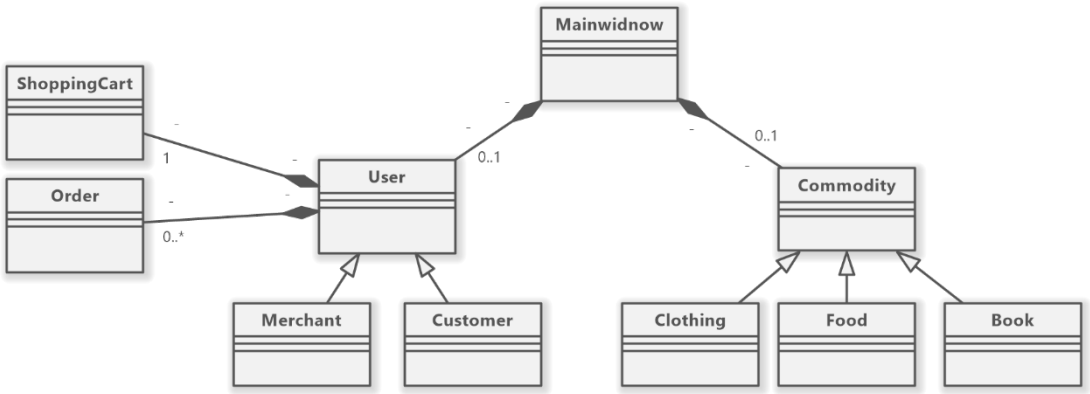
```
 */

class CommodityDelegate : public QStyledItemDelegate {
public:
    CommodityDelegate(QObject *parent = nullptr) : QStyledItemDelegate(parent) {}
    void paint(QPainter *painter, const QStyleOptionViewItem &option, const QModelIndex
&index) const;// 自定义绘制商品卡片
    QSize sizeHint(const QStyleOptionViewItem &option, const QModelIndex &index) const;
};//返回商品卡片的固定尺寸
```

3.1.3 业务逻辑层（BLL）

3.1.3.1 功能介绍

业务逻辑层提供了一系列方法给 UIL 进行调用，主要是 user 类等。这些类的方法负责构建数据包的内容，最后调用 NL 层的方法将内容发送出去。

3.1.3.2 类之间的关系



这里的实心菱形箭头表示组合（强包含），也就是说 mainwindow 里包含（含有指针）0 或 1 个 user 类和 commodity 类，每一个 user 类维护（含有指针或者是列表）着一个 shoppingcart 类和多个 order 类。
user 类派生出 merchant 和 customer 类，commodity 类派生出 clothing，food 和 book 类。这些类提供了相关的操作方法，这些方法一部分用于维护本地内存中的数据，一部分构造报文发送至服务器，请求服务器操作。

3.1.3.3 详细类介绍

user 类的详细介绍如下，所有的方法和成员变量都加以注释：

```
/**
 * @brief 用户类，处理用户相关的所有业务逻辑
 */
class User:public QObject
{
    Q_OBJECT
public:
    // 构造析构
    User(); // 默认构造函数
    User(const QString& name, double balance, const QString& type, const QString&
password = ""); // 兼容旧版本的构造函数
    User(const QString& name, const QString& password, const QString& type, unsigned long
long balance); // 推荐使用的构造函数
    ~User(); // 析构函数

    // 需要向服务器发送数据的操作
    void userLogin(const QString& username, const QString& password); // 用户登录
    void userRegister(const QString& username, const QString& password, const QString&
type); // 用户注册
    void purchase(unsigned long long totalPrice,const QString& username,unsigned int
id,unsigned int purchaseQuantity,const QString& sourceStoreName); // 立即购买商品
    void changePassword(const QString& newPassword); // 修改密码
    void recharge(unsigned long long amount); // 用户充值

    // 用户获取方法
    unsigned long long getBalance() const; // 获取余额(分为单位)
    QString getType() const; // 获取用户类型
    QString getName() const; // 获取用户名
    QString getPassword() const; // 获取密码

    // 用户设置方法
    void setBalance(double balance); // 设置余额(元为单位)
    void setBalanceWithout100(double balance); // 设置余额(分为单位)

    // 购物车操作
    ShoppingCart* getShoppingCart() const; // 获取购物车对象
    void setShoppingCart(ShoppingCart* cart) { shoppingCart = cart; } // 设置购物车对
象

    // 订单操作
    void addOrder(Order* order); // 添加订单
```

```
    void removeOrder(unsigned int orderId); // 移除订单
    QList<Order*> getOrders() const; // 获取所有订单
    Order* getOrderById(unsigned int orderId) const; // 根据 ID 获取订单
    void clearOrders(); // 清空所有订单

protected:
    QString name; // 用户名
    QString password; // 密码
    unsigned long long balance; // 余额(分为单位)
    QString type; // 用户类型(Customer/Merchant)

    ShoppingCart* shoppingCart=nullptr; // 购物车对象指针
    QList<Order*> orders; // 订单列表
};

/**
 * @brief 客户类，继承自 User，经验教训处说明了这里没有内容的原因。
 */
class Customer:public User
{
};

/**
 * @brief 商家类，继承自 User，经验教训处说明了这里没有内容的原因。
 */
class Merchant:public User
{
};
```

commodity 类的详细说明如下，所有的方法和成员变量都加以注释。

```
/**
 * @brief 商品基类，处理商品相关的所有业务逻辑
 */
class Commodity
{
public:
    // 构造析构
    Commodity(); // 默认构造函数
    Commodity(unsigned int id, const QString& name, const QString& type, const QString&
description,
              unsigned long long originalPrice, unsigned int stock, const QString&
sourceStoreName,
              float discount, float uniDiscount); // 参数构造函数
    virtual ~Commodity()=default; // 虚析构函数
```

```
    // 数据发送方法
    void send_fetchAllInformation(unsigned int id,int page); // 获取商品详细信息
    void updateCommodity(unsigned int id, const QString& name, const QString& type, const
QString& description,
                         unsigned long long originalPrice, unsigned int stock, const
QString& sourceStoreName,
                         double discount, double uniDiscount); // 更新商品信息(double 版
本)
    void newCommodity(const QString& name, const QString& type, const QString&
description,
                         unsigned long long originalPrice, unsigned int stock, const
QString& sourceStoreName,
                         double discount, double uniDiscount); // 创建新商品(double 版本)
    static void setUniDiscount(float uniDiscount, const QString &sourceStoreName, const
QString &type); // 设置统一折扣

    // 网络操作
    static void fetchAllInfoOfCommodityFromServer(unsigned int commodityId, const
QString& context); // 从服务器获取商品信息
    static void purchaseFromServer(const QString& buyerName, unsigned int commodityId,
unsigned int quantity); // 服务器购买请求
    void updateCommodity(unsigned int id, const QString& name, const QString& type, const
QString& description,
                         unsigned long long originalPrice, unsigned int stock, const
QString& sourceStoreName,
                         float discount, float uniDiscount); // 更新商品信息(float 版本)
    void newCommodity(const QString& name, const QString& type, const QString&
description,
                      unsigned long long originalPrice, unsigned int stock, const QString&
sourceStoreName,
                      float discount, float uniDiscount); // 创建新商品(float 版本)
    static void setUniDiscountFromServer(const QString& sourceStoreName, float
uniDiscount, const QString& type); // 向服务器设置统一折扣
    void deleteCommodityFromServer(); // 从服务器删除商品

    // 获取方法
    unsigned int getId() const { return id; } // 获取商品 ID
    QString getName() const { return name; } // 获取商品名称
    QString getType() const { return type; } // 获取商品类型
    QString getDescription() const { return description; } // 获取商品描述
    unsigned long long getOriginalPrice() const { return originalPrice; } // 获取原价
(分为单位)
    unsigned int getStock() const { return stock; } // 获取库存数量
```

```
    QString getSourceStoreName() const { return sourceStoreName; } // 获取来源商店名
    float getDiscount() const { return discount; } // 获取个人折扣
    float getUniDiscount() const { return uniDiscount; } // 获取统一折扣

    // 设置方法
    void setStock(unsigned int newCommodityStock); // 设置库存数量

    // 计算价格
    virtual unsigned long long Getprice() const; // 计算最终价格(含折扣)

protected:
    unsigned int id; // 商品 ID
    QString name; // 商品名称
    QString type; // 商品类型(Book/Food/Clothing)
    QString description; // 商品描述
    unsigned long long originalPrice; // 原价(分为单位)
    unsigned int stock; // 库存数量
    QString sourceStoreName; // 来源商店名
    float discount; // 个人折扣(0-1 之间)
    float uniDiscount; // 统一折扣(0-1 之间)
};

/**
 * @brief 书籍类，继承自 Commodity
 */
class Book : public Commodity
{
public:
    Book() : Commodity() { type = "Book"; } // 默认构造函数
    ~Book() override= default; // 析构函数

    Book(unsigned int id, const QString &name, const QString &description,
        unsigned long long originalPrice, unsigned int stock,
        const QString &sourceStoreName, float discount) : Commodity(); // 参数构造函
数
    unsigned long long Getprice() const override; // 重写价格计算方法

private:
};

/**
 * @brief 食品类，继承自 Commodity
 */
class Food : public Commodity
```

```
{
public:
    Food() : Commodity() { type = "Food"; } // 默认构造函数
    ~Food() override= default; // 析构函数

    Food(unsigned int id, const QString &name, const QString &description,
        unsigned long long originalPrice, unsigned int stock,
        const QString &sourceStoreName, float discount) : Commodity(); // 参数构造函
数

    unsigned long long Getprice() const override; // 重写价格计算方法

private:
};

/**
 * @brief 服装类，继承自 Commodity
 */
class Clothing : public Commodity
{
public:
    Clothing() : Commodity() { type = "Clothing"; } // 默认构造函数
    ~Clothing() override= default; // 析构函数

    Clothing(unsigned int id, const QString &name, const QString &description,
            unsigned long long originalPrice, unsigned int stock,
            const QString &sourceStoreName, float discount) : Commodity(); // 参数构
造函数
    unsigned long long Getprice() const override; // 重写价格计算方法

private:
};
```

order 类的详细介绍，所有的方法和成员变量都加以注释。

```
/**
 * @brief 订单类，处理订单相关的所有业务逻辑
 */
class Order {
public:
    // 构造析构
    Order(); // 默认构造函数
    Order(const QString& userName, const QList<QVariantMap>& items, unsigned long long
totalPrice, const QString& state = "unPaid"); // 参数构造函数
    ~Order(); // 析构函数
```

```
    // 获取方法
    unsigned int getOrderId() const; // 获取订单 ID
    QString getUserName() const; // 获取用户名
    QList<QVariantMap> getItems() const; // 获取订单商品列表
    unsigned long long getTotalPrice() const; // 获取订单总价(分为单位)
    QString getState() const; // 获取订单状态(unPaid/paid/cancelled)
    QString getOrderTime() const; // 获取订单创建时间

    // 设置方法
    void setState(const QString& state); // 设置订单状态
    void setOrderId(unsigned int id); // 设置订单 ID
    void setOrderTime(const QString& time); // 设置订单时间

    // 网络操作
    static void fetchOrdersFromServer(const QString& userName); // 从服务器获取用户订单
列表
    static void deleteOrderFromServer(const QString& userName, unsigned int orderId);
// 从服务器删除订单
    static void createOrderFromServer(const QString& userName, const
QList<QVariantMap>& orderItems, unsigned long long totalPrice); // 在服务器创建订单
    static void payOrderFromServer(const QString& userName, unsigned int orderId); //
向服务器发送支付请求

private:
    unsigned int orderId; // 订单唯一标识符
    QString userName; // 订单所属用户名
    QList<QVariantMap> items; // 订单商品列表(包含商品 ID、数量、价格等信息)
    unsigned long long totalPrice; // 订单总价(分为单位)
    QString state; // 订单状态(unPaid 未支付/paid 已支付/cancelled 已取消)
    QString orderTime; // 订单创建时间
};
```

shoppingcart 类的详细介绍，所有的方法和成员变量都加以注释。

```
/**
 * @brief 购物车项目结构体
 */
struct CartItem {
    unsigned int id; // 购物车项目 ID
    unsigned int commodityId; // 商品 ID
    int quantity; // 商品数量
};

/**
```

```
 * @brief 购物车类，处理购物车相关的所有业务逻辑
 */
class ShoppingCart : public QObject
{
    Q_OBJECT

public:
    // 构造析构
    ShoppingCart(const QString& userName, QObject *parent); // 构造函数
    ~ShoppingCart(); // 析构函数

    // 购物车操作(带错误信息版本)
    bool addItem(unsigned int commodityId, int quantity, QString& errorMessage); // 添加商品到购物车
    bool removeItem(unsigned int id, QString& errorMessage); // 从购物车移除项目(按项目ID)
    void updateItemQuantity(unsigned int id, int quantity, QString& errorMessage); // 更新购物车项目数量
    bool clear(QString &errorMessage); // 清空购物车(带错误信息)

    // 购物车操作(简化版本)
    void removeItem(unsigned int commodityId); // 移除项目(按商品 ID)
    void removeItemByCommodityId(unsigned int commodityId); // 按商品 ID 删除项目
    void clear(); // 清空购物车

    // 数据操作
    QList<CartItem> getItems() const; // 获取所有购物车项目
    bool saveToDatabase() const; // 保存到数据库
    void loadFromDatabase(); // 从数据库加载
    void updateFromServerResponse(const QJsonArray &updatedCartItems); // 从服务器响应更新购物车

    // 获取方法
    QString getUserName() const { return userName; } // 获取用户名

private slots:
    void onCartFetched(bool success, const QString &message, const QJsonArray &cartItems); // 购物车数据获取完成槽函数

signals:
    void cartUpdated(bool success, const QString &message, const QList<CartItem> &items, const QJsonArray &cartItemsJson); // 购物车更新信号

private:
```

```
    QString userName; // 购物车所属用户名
    QList<CartItem> items; // 购物车项目列表
};
```

3.1.4 网络通信层（NL）

3.1.4.1 功能介绍

网络通信层一方面用于发送报文，另一方面用于处理判断服务端发送的回答报文，并向 BLL 层或者是 UIL 层发出信号。

3.1.4.2 详细类介绍

由于网络通信层只有一个类，即 HandleMessage 类，该类在 main 函数中初始化为全局变量。便于向其他类发送信号。
详细实现如下：

```
/**
 * @brief 网络消息处理类，负责客户端与服务器的通信
 */
class HandleMessage:public QObject
{
    Q_OBJECT
public:
    explicit HandleMessage(QObject *parent = nullptr); // 构造函数

    // 静态方法：发送带长度前缀的 JSON 数据
    static bool sendJsonData(QTcpSocket* socket, const QJsonObject& jsonObject); // 发送 JSON 数据到服务器

signals:
    // 基础通信信号
    void serverResponse(const QJsonObject& response); // 通用服务器响应信号

    // 用户认证相关信号
    void registerResponse(bool success,QString message); // 注册响应信号
    void loginResponse(bool success,const QString& message,const QString& username,const QString& password,const QString& type,unsigned long long balance); // 登录响应信号
    void changePasswordResponse(bool success, const QString& message); // 修改密码响应信号
```

```
    void rechargeResponse(bool success, const QString& message, unsigned long long
newBalance = 0); // 充值响应信号

    // 商品相关信号
    void commoditiesResponseReceived(const QJsonDocument& responseDoc, const QString&
context); // 商品列表响应信号
    void fetchAllInfoOfCommodityResponse(bool success, const QString& message, const
QVariantMap& commodityData); // 主页商品详情响应信号
    void fetchAllInfoOfCommodityResponse_page3(bool success, const QString& message,
const QVariantMap& commodityData); // 商品管理页详情响应信号
    void commodityUpdateResponse(bool success, const QString& message); // 商品更新响
应信号
    void commodityNewResponse(bool success, const QString& message); // 新增商品响应信
号
    void deleteCommodityResponse(bool success, const QString& message, unsigned int
deletedCommodityId); // 删除商品响应信号
    void setUniDiscountResponse(bool success, const QString& message, float uniDiscount,
const QString& sourceStoreName, const QString& type); // 设置统一折扣响应信号

    // 购买相关信号
    void purchaseResponse(bool success, const QString& message, unsigned long long
newBuyerBalance = 0, unsigned int newCommodityStock = 0); // 购买响应信号

    // 购物车相关信号
    void cartFetched(bool success, const QString &message, const QJsonArray &cartItems);
// 获取购物车响应信号
    void updateCartItemQuantityResponse(bool success, const QString& message, const
QString& action, const QJsonArray& updatedCartItems); // 更新购物车数量响应信号
    void addToCartResponse(bool success, const QString& message, const QJsonArray&
updatedCartItems); // 加入购物车响应信号
    void removeCartItemResponse(bool success, const QString& message, const QString&
action, const QJsonArray& updatedCartItems); // 移除购物车项目响应信号
    void clearCartResponse(bool success, const QString& message, const QString& action,
const QJsonArray& updatedCartItems); // 清空购物车响应信号

    // 订单相关信号
    void fetchOrdersResponse(bool success, const QString& message, const QJsonArray&
orders); // 获取订单列表响应信号
    void deleteOrderResponse(bool success, const QString& message, unsigned int
deletedOrderId); // 删除订单响应信号
    void validateOrderItemsResponse(const QString& status, const QString& message, const
QJsonArray& validatedItems, unsigned long long totalPrice, const QString& changes); //
订单验证响应信号
    void createOrderResponse(bool success, const QString& message, unsigned long long
```

```
newBalance, unsigned long long totalPrice); // 创建订单响应信号
    void payOrderResponse(bool success, const QString& message, unsigned int orderId,
unsigned long long paidAmount, unsigned long long newBalance); // 支付订单响应信号

private slots:
    void onReadyRead(); // 处理套接字可读数据槽函数

private:
    QByteArray m_buffer; // 接收缓冲区，用于存储不完整的消息
    quint32 m_nextBlockSize=0; // 下一个消息的预期长度，用于消息分包处理

};
```

3.1.5 各层通信的一个实例

仅举更新购物车操作一例，以详细说明客户端的基本逻辑。
首先看下面的几个声明，这些声明构成了客户端基本逻辑的核心。

表现层（UIL）	①	private:	void MainWindow::updateCartDisplay()
	②	private slots:	void MainWindow::onCartUpdated(bool success, const QString &message, const QList<CartItem> &cartListItems, const QJsonArray &cartItemsJson)
业务逻辑层（BLL）	③	public:	void loadFromDatabase();
	④	private slots:	void onCartFetched(bool success, const QString &message, const QJsonArray &cartItems);
	⑤	signals:	void cartUpdated(bool success, const QString &message, const QList<CartItem> &items, const QJsonArray &cartItemsJson);
网络通信层（NL）	⑥	public:	static bool sendJsonData(QTcpSocket* socket, const QJsonObject& jsonObject);
	⑦	signals:	void cartFetched(bool success, const QString &message, const QJsonArray &cartItems);

①updateCartDisplay 是 mainwindow 的一个方法，用于点击购物车页面时，刷新购物车，该函数只负责基本的判断购物车对象是否存在，若存在调用购物车类的函数 loadFromDatabase
③loadFromDatabase 被调用，该函数整理好数据包的内容，调用网络通信层的函数 sendJsonData 发送出去。
⑥sendJsonData 函数被调用，发送出去数据包。
⑦cartFetched 是网络通信层收到回应包时发出的信号。该信号与购物车类的 onCartFetched 函数相连接。
④onCartFetched 收到触发信号，自动调用，该函数更新本地内存中的购物车的数据，然后发送信号 cartUpdated
⑤cartUpdated 是购物车类发出的信号，该信号与 UI 层 mainwindow 窗口的 onCartUpdated 方法相连接。

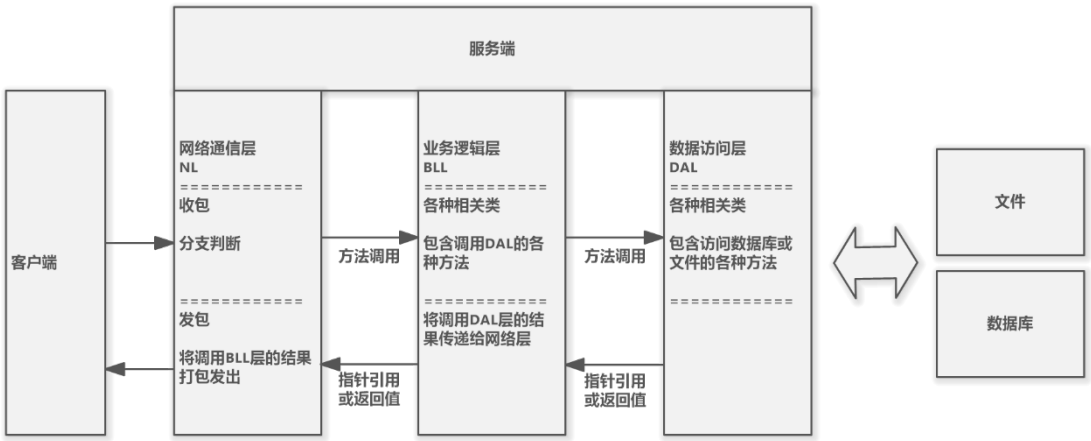
②onCartUpdated 收到触发信号，自动调用，更新了 UI 层的购物车表格信息。

整个客户端遵循如上逻辑，构成了整个客户端的核心逻辑。

3.2 子系统服务端详细设计

3.2.1 总体介绍

服务端分为网络通信层（NL），业务逻辑层（BLL），数据访问层（DAL）三层。其中网络通信层能够处理客户端发送的各种请求报文，通过方法调用 BLL 层，BLL 层调用 DAL 层来实现，并根据处理之后返回的错误类型来发送应答报文。

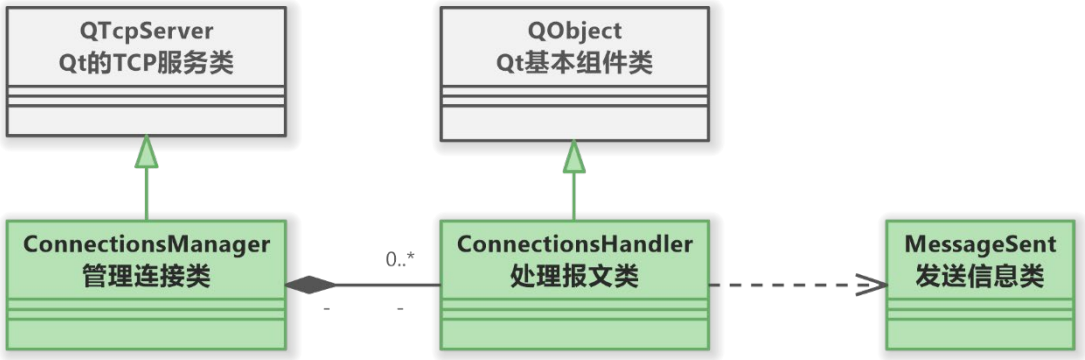


3.2.2 网络通信层（NL）

3.2.2.1 功能介绍

网络通信层的主要任务有三个：一是完成连接的建立，即每当新的客户端请求建立连接时，都能够为其创建出一个连接，并分配一个连接处理对象。二是报文处理，连接处理对象能够根据发送过来的不同类型的信息进行不同的应答处理。三是发送回答报文。

3.2.2.2 类之间的关系



这里的管理连接类继承自 Qt 的 QTcpServer 类，处理报文类继承自 QObject 类。管理连接类提供了处理新连接的方法，建立 TCP 连接，并为每一个连接分配一个处理报文对象。处理报文类根据报文类型的不同，为每一种请求提供了一种处理方法，最后调用发送信息类来发送请求，该类仅有一个静态函数，故处理报文类和发送信息类之间是依赖关系，用虚线表示。

3.2.2.3 详细类介绍

管理连接类的详细信息如下，所有的方法和成员变量都加以注释。

```
/**
 * @brief 连接管理器类，负责管理 TCP 服务器和客户端连接
 */
class ConnectionsManager : public QTcpServer
{
    Q_OBJECT
public:
    explicit ConnectionsManager(QObject *parent = nullptr); // 构造函数
    void startServer(quint16 port); // 启动 TCP 服务器

protected:
    void incomingConnection(qintptr socketDescriptor) override; // 处理新客户端连接

private slots:
    void onClientHandlerDisconnected(ConnectionsHandler* handler); // 处理客户端断开连
接

private:
    QList<ConnectionsHandler*> m_clientHandlers; // 存储所有活跃的客户端连接处理器
};
```

发送信息类的详细信息如下，所有的方法和成员变量都加以注释。

```
/**
 * @brief 消息发送工具类，提供网络数据发送功能
 */
class MessageSent
{
public:
    static bool sendJsonData(QTcpSocket* socket, const QJsonObject& jsonObject); // 发送 JSON 数据到客户端
};
```

处理报文类的详细信息如下，所有的方法和成员变量都加以注释。

```
/**
 * @brief 客户端连接处理器类，负责处理单个客户端的所有业务请求
 */
class ConnectionsHandler : public QObject
{
    Q_OBJECT
public:
    // 构造析构
    explicit ConnectionsHandler(QTcpSocket *socket, QObject *parent = nullptr); // 构造函数，传入已连接的套接字
    ~ConnectionsHandler(); // 析构函数

signals:
    void clientDisconnected(ConnectionsHandler* handler); // 客户端断开连接信号

private slots:
    // 网络事件处理
    void onReadyRead(); // 处理套接字可读数据
    void onDisconnected(); // 处理套接字断开连接
    void onErrorOccurred(QAbstractSocket::SocketError socketError); // 处理套接字错误

private:
    // 网络通信相关
    QTcpSocket *m_socket; // 该处理器管理的套接字
    QByteArray m_buffer; // 接收缓冲区，用于处理分包数据
    quint32 m_nextBlockSize; // 下一个消息的预期长度

    // 业务对象
    Commodity m_commodity; // 商品业务对象
    User m_user; // 用户业务对象
    ShoppingCart cart; // 购物车业务对象

    // 核心业务处理
```

```
    QJsonObject processRequest(const QJsonObject& request); // 处理客户端请求的主入口

    // 用户认证相关处理函数
    QJsonObject handleRegisterRequest(const QJsonObject& request); // 处理用户注册请求
    QJsonObject handleLoginRequest(const QJsonObject& request); // 处理用户登录请求
    QJsonObject handleChangePasswordRequest(const QJsonObject& request); // 处理修改密
码请求
    QJsonObject handleRechargeRequest(const QJsonObject& request); // 处理用户充值请求

    // 商品相关处理函数
    QJsonObject handleQueryProductRequest(const QJsonObject& request); // 处理商品查询
请求
    QJsonObject handleFetchCommoditiesRequest(const QJsonObject& request); // 处理获
取商品列表请求
    QJsonObject handleFetchAllInfoOfCommodityRequest(const QJsonObject& request); //
处理获取商品详细信息请求
    QJsonObject handleUpdateCommodityRequest(const QJsonObject& request); // 处理更新
商品信息请求
    QJsonObject handleNewCommodityRequest(const QJsonObject& request); // 处理新增商品
请求
    QJsonObject handleDeleteCommodityRequest(const QJsonObject& request); // 处理删除
商品请求
    QJsonObject handleSetUniDiscountRequest(const QJsonObject& request); // 处理设置统
一折扣请求
    QJsonObject handleGetUserUniDiscountRequest(const QJsonObject& request); // 处理
获取用户统一折扣请求

    // 购买相关处理函数
    QJsonObject handlePurchaseRequest(const QJsonObject& request); // 处理直接购买请求

    // 购物车相关处理函数
    QJsonObject handleFetchCartRequest(const QJsonObject &request); // 处理获取购物车
请求
    QJsonObject handleAddToCartRequest(const QJsonObject& request); // 处理添加商品到
购物车请求
    QJsonObject handleUpdateCartItemQuantityRequest(const QJsonObject &request); // 处
理更新购物车商品数量请求
    QJsonObject handleRemoveCartItemRequest(const QJsonObject& request); // 处理移除购
物车商品请求
    QJsonObject handleClearCartRequest(const QJsonObject& request); // 处理清空购物车
请求

    // 订单相关处理函数
    QJsonObject handleFetchOrdersRequest(const QJsonObject& request); // 处理获取订单
```

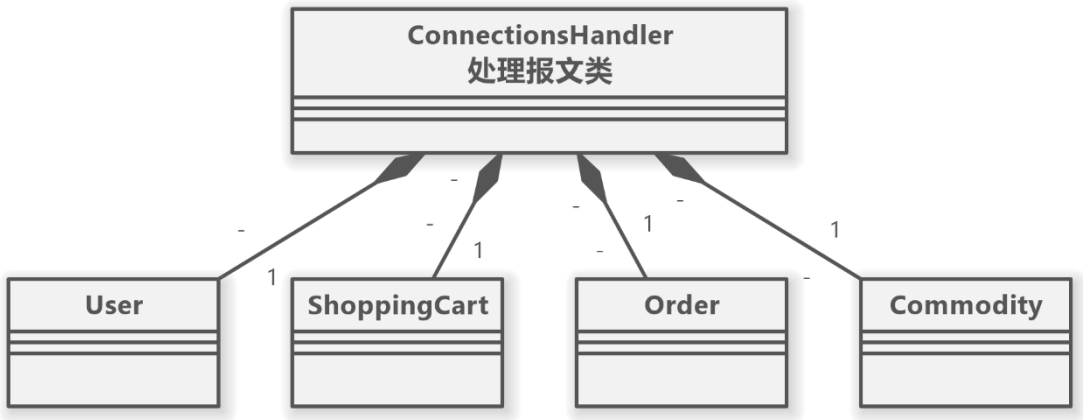
```
列表请求
    QJsonObject handleValidateOrderItemsRequest(const QJsonObject& request); // 处理订单项目验证请求
    QJsonObject handleCreateOrderRequest(const QJsonObject& request); // 处理创建订单请求
    QJsonObject handlePayOrderRequest(const QJsonObject& request); // 处理支付订单请求
    QJsonObject handleDeleteOrderRequest(const QJsonObject& request); // 处理删除订单请求
};
```

3.2.3 业务逻辑层（BLL）

3.2.3.1 功能介绍

业务逻辑层向网络通信层提供了处理方法的接口，这里主要是由各种类组成，这些类通过调用数据访问层的方法来进行数据的处理。

3.2.3.2 类之间的关系



这里给出四个类，分别为 User 类，ShoppingCart 类，Order 类 Commodity 类。处理报文类中包含着这四个类。值得注意的是，这里的四个类并没有像客户端那样有很多的成员变量，几乎直接调用 DAL 进行数据库操作，且这里的类也没有什么继承关系。

3.2.3.3 详细类介绍

User 类详细介绍如下，所有的方法和成员变量都加以注释。

```
/**
 * @brief 服务器端用户业务逻辑类
 */
class User
{
public:
    User(); // 默认构造函数

    // 用户数据库操作
    static User* createFromDatabaseByName(const QString& name); // 从数据库根据用户名创建用户对象
    bool saveToDatabase() const; // 保存用户信息到数据库
    static bool transferToMerchant(const QString& merchantName, unsigned long long amount, QString& errorMessage); // 向商家转账
    QVariantMap makePurchase(const QString& buyerUsername, unsigned int commodityId, unsigned int purchaseQuantity, const QString& sellerStoreName, unsigned long long totalPrice); // 执行购买操作
    QVariantMap rechargeBalance(const QString& username, unsigned long long amount); // 用户充值操作

    // 用户获取方法
    unsigned long long getBalance() const; // 获取用户余额
    QString getType() const; // 获取用户类型
    QString getName() const; // 获取用户名
    QString getPassword() const; // 获取用户密码

    // 用户设置方法
    void setBalance(double balance); // 设置用户余额
    QVariantMap changePassword(const QString& username, const QString& newPassword); // 修改用户密码

    // 登录注册
    LoginErrorType userLogin(const QString& username, const QString& password); // 用户登录验证
    RegisterErrorType userRegister(const QString& username, const QString& password, const QString& type); // 用户注册

    // 购买
    bool purchase(unsigned long long totalPrice, QString& errorMessage); // 购买商品(扣除余额)

    // 购物车操作
    //ShoppingCart* getShoppingCart() const; // 获取购物车(已注释)
```

```
    // 订单操作
    //QList<Order*> getOrders() const; // 获取订单列表(已注释)
    //void addOrder(Order* order); // 添加订单(已注释)
    void removeOrder(unsigned int orderId); // 移除订单

protected:
    QString name; // 用户名
    QString password; // 用户密码
    unsigned long long balance; // 用户余额(分为单位)
    QString type; // 用户类型(Customer/Merchant)
    UserDAO m_userDAO; // 用户数据访问对象
    CommodityDAO m_commodityDAO; // 商品数据访问对象
};
```

Commodity 类介绍如下，所有的方法和成员变量都加以注释。

```
/**
 * @brief 服务器端商品业务逻辑类
 */
class Commodity
{
public:
    Commodity(); // 构造函数

    // 商品信息操作
    QVariantMap getCommodityDetails(unsigned int commodityId); // 根据商品 ID 获取详细
信息
    QVariantMap updateCommodity(unsigned int id, const QString& name, const QString&
type, const QString& description,
                                unsigned long long originalPrice, unsigned int stock,
const QString& sourceStoreName,
                                double discount, double uniDiscount); // 更新商品信息
    QVariantMap newCommodity(const QString& name, const QString& type, const QString&
description,
                             unsigned long long originalPrice, unsigned int stock, const
QString& sourceStoreName,
                             double discount, double uniDiscount); // 创建新商品

    // 商品管理操作
    QVariantMap setCommodityUniDiscount(const QString& sourceStoreName, const QString&
type, float uniDiscount); // 设置统一折扣
    static bool deleteCommodity(unsigned int commodityId, const QString&
sourceStoreName); // 删除商品

private:
```

```
    CommodityDAO m_commodityDAO; // 商品数据访问对象
};
```

Order 类介绍如下，所有的方法和成员变量都加以注释。

```
/**
 * @brief 服务器端订单业务逻辑类
 */
class Order {
public:
    // 构造析构
    Order(); // 默认构造函数
    Order(const QString& userName, const QList<QVariantMap>& items, unsigned long long totalPrice, const QString& state = "unPaid"); // 参数构造函数
    ~Order(); // 析构函数

    // 获取方法
    unsigned int getOrderId() const; // 获取订单 ID
    QString getUserName() const; // 获取用户名
    QList<QVariantMap> getItems() const; // 获取订单商品列表
    unsigned long long getTotalPrice() const; // 获取订单总价(分为单位)
    QString getState() const; // 获取订单状态
    QString getOrderTime() const; // 获取订单创建时间

    // 设置方法
    void setState(const QString& state); // 设置订单状态
    void setOrderId(unsigned int id); // 设置订单 ID
    void setOrderTime(const QString& time); // 设置订单时间

    // 数据库操作（服务端）
    bool saveToDatabase() const; // 保存订单到数据库
    static QList<Order> fetchOrdersByUser(const QString& userName); // 根据用户名获取订单列表
    bool updateStateInDatabase() const; // 更新订单状态到数据库

private:
    unsigned int orderId; // 订单 ID，自增 INT
    QString userName; // 关联的用户名
    QList<QVariantMap> items; // 订单中的商品列表
    unsigned long long totalPrice; // 总价（以分为单位）
    QString state; // 订单状态：unPaid, paid, shipped, completed, canceled
    QString orderTime; // 订单创建时间
};
```

ShoppingCart 类介绍如下，所有的方法和成员变量都加以注释。

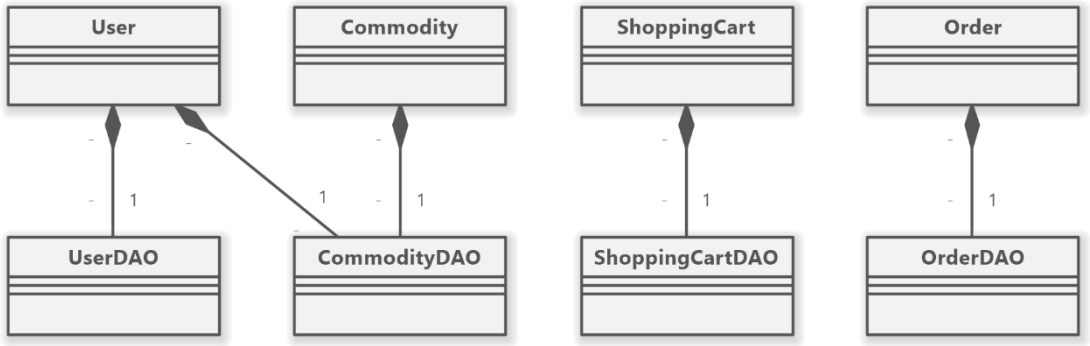
```
/**
 * @brief 服务器端购物车业务逻辑类
 */
class ShoppingCart : public QObject
{
    Q_OBJECT

public:
    ShoppingCart(const QString &username); // 构造函数，指定用户名

    // 购物车数据操作
    bool fetchCartItems(QJsonArray &cartItems, QString &errorMessage); // 获取购物车商品列表
    void setUsername(const QString &username); // 设置购物车所属用户名

    // 购物车商品操作
    bool removeItem(unsigned int cartItemId, QString &errorMessage); // 移除购物车商品项
    bool updateItemQuantity(unsigned int cartItemId, int quantity, QString &errorMessage); // 更新商品数量
    bool addItem(unsigned int commodityId, int quantity, QString &errorMessage); // 添加商品到购物车
    bool clear(QString &errorMessage); // 清空购物车

private:
    QString username; // 购物车所属用户名
};
```

3.2.4 数据访问层（DAL）

3.2.4.1 功能介绍

数据访问层提供了访问数据库或者是文件的一系列方法。也以类的方式展现出来。

3.2.4.2 类之间的关系



上面的是业务逻辑层的对象，下面的是数据访问层的对象，基本是一一对应的包含关系。而 User 类之所以维护商品数据访问对象，是因为用户类进行购买操作时需要验证，此时需要抓取商品类的全部信息，故直接使用商品数据访问类，其它的类虽然也有使用到商品类的信息，但是却是直接使用 MySQL 的操作原语。这里的设计受限于时间精力也许不太恰当。

3.2.4.3 详细类介绍

userDAO 类如下，同时由于 userDAO 使用文件操作，这里设计一些文件相关的实现，所有的方法和成员变量都加以注释。

```
// 登录错误类型枚举
enum class LoginErrorType {
    NoError,
    UsernameInexists,
    WrongPassword,
    FileError
};

// 注册错误类型枚举
enum class RegisterErrorType {
    NoError,
    UsernameAlreadyExists,
    FileError
};

/**
 * @brief 用户数据访问对象，负责用户数据的持久化操作
 */
class UserDAO {
private:
    static QString dataFilePath; // 数据文件路径
```

```
    static unsigned int nextUserId; // 下一个用户 ID
    static QMutex fileMutex; // 线程安全保护

    // 私有文件操作方法
    static bool loadUsersFromFile(QList<UserData>& users); // 从文件加载用户数据
    static bool saveUsersToFile(const QList<UserData>& users); // 保存用户数据到文件
    static bool ensureDataFileExists(); // 确保数据文件存在
    static unsigned int getNextUserId(); // 获取下一个用户 ID

public:
    // 初始化文件系统
    static bool initializeFileSystem(const QString& dataDirectory = "data"); // 初始
化数据目录

    // 用户认证相关方法
    static LoginErrorType userLogin(const QString& username, const QString& password,
QString* outType, unsigned long long* outBalance); // 用户登录验证
    static RegisterErrorType userRegister(const QString& username, const QString&
password, const QString& type); // 用户注册
    static bool updateUserPassword(const QString& username, const QString& newPassword);
// 更新用户密码

    // 用户信息查询方法
    static bool getUserIdAndBalanceByUsername(const QString& username, unsigned int*
outId, unsigned long long* outBalance); // 根据用户名获取 ID 和余额
    static unsigned int getUserIdByUsername(const QString& username); // 根据用户名获
取用户 ID
    static bool userExists(const QString& username); // 检查用户是否存在
    static UserData getUserByUsername(const QString& username); // 根据用户名获取用户数
据
    static UserData getUserById(unsigned int userId); // 根据用户 ID 获取用户数据
    static QList<UserData> getAllUsers(); // 获取所有用户数据

    // 余额管理方法
    static bool updateUserBalance(unsigned int userId, qint64 amount, unsigned long long*
outNewBalance); // 更新用户余额

    // 统一折扣管理方法
    static bool getUserUniDiscount(const QString& username, const QString& type, float*
outUniDiscount); // 获取用户统一折扣
    static bool setUserUniDiscount(const QString& username, const QString& type, float
uniDiscount); // 设置用户统一折扣
    static QMap<QString, float> getUserAllUniDiscounts(const QString& username); // 获
取用户所有统一折扣
```

```
};
```

CommodityDAO 类如下，所有的方法和成员变量都加以注释。

```
/**
 * @brief 商品数据访问对象，负责商品数据的持久化操作
 */
class CommodityDAO
{
public:
    // 商品查询方法
    bool FetchAllCommodities(const QString& storeName,QJsonArray& commoditiesArray);
// 获取指定商店的所有商品
    QVariantMap fetchCommodityDetailsById(unsigned int commodityId); // 根据商品 ID 获
取详细信息

    // 库存管理方法
    bool getCommodityStock(unsigned int commodityId, unsigned int* outStock); // 获取
商品库存数量
    bool updateCommodityStock(unsigned int commodityId, qint64 quantityChange, unsigned
int* outNewStock = nullptr); // 更新商品库存(正数增加，负数减少)

    // 商品信息管理方法
    bool updateCommodity(unsigned int id, const QString& name, const QString& type, const
QString& description,
                         unsigned long long originalPrice, unsigned int stock, const
QString& sourceStoreName,
                         float discount, float uniDiscount); // 更新商品信息
    bool newCommodity(const QString& name, const QString& type, const QString&
description,
                         unsigned long long originalPrice, unsigned int stock, const
QString& sourceStoreName,
                         float discount, float uniDiscount); // 创建新商品
    bool deleteCommodity(unsigned int commodityId, const QString& sourceStoreName); //
删除商品

    // 折扣管理方法
    bool updateCommodityUniDiscount(const QString& sourceStoreName, const QString&
type, float uniDiscount); // 更新商品统一折扣
};
```

ShoppingCart 类如下，所有的方法和成员变量都加以注释。

```
/**
 * @brief 购物车数据访问对象，负责购物车数据的持久化操作
 */
```

```
class ShoppingCartDAO {
public:
    // 购物车数据操作
    bool fetchCartItems(const QString &username, QJsonArray &cartItems, QString
&errorMessage); // 获取用户购物车商品列表
    bool addCartItem(const QString &username, unsigned int commodityId, int quantity,
QString &errorMessage); // 添加商品到购物车
    bool removeCartItem(const QString &username, unsigned int cartItemId, QString
&errorMessage); // 移除购物车商品项
    bool updateCartItemQuantity(const QString &username, unsigned int cartItemId, int
quantity, QString &errorMessage); // 更新购物车商品数量
    bool clearCart(const QString &username, QString &errorMessage); // 清空用户购物车
};
```

OrderDAO 类如下，所有的方法和成员变量都加以注释。

```
/**
 * @brief 订单数据访问对象，负责订单数据的持久化操作
 */
class OrderDAO {
public:
    OrderDAO(); // 构造函数
    ~OrderDAO(); // 析构函数

    // 订单基本操作
    bool saveOrder(const Order& order); // 保存订单到数据库
    QList<Order> fetchOrdersByUser(const QString& userName); // 根据用户名获取订单列表
    Order getOrderById(unsigned int orderId); // 根据订单 ID 获取订单
    bool deleteOrder(unsigned int orderId); // 删除订单
    bool deleteOrderAndRestoreStock(unsigned int orderId, const QString& userName); //
删除订单并恢复库存

    // 订单状态管理
    bool updateOrderState(unsigned int orderId, const QString& state); // 更新订单状态

private:
    // 订单项管理
    bool saveOrderItems(unsigned int orderId, const QList<QVariantMap>& items); // 保
存订单项到数据库
    QList<QVariantMap> fetchOrderItems(unsigned int orderId); // 获取订单项
};
```

3.2.5 各层通信的一个实例

仍旧举购物车更新的例子，服务端的函数如下：

网络通信层（NL）	①	private	QJsonObject handleFetchCartRequest(const QJsonObject &request);
	②	public	static bool sendJsonData(QTcpSocket* socket, const QJsonObject& jsonObject);
业务逻辑层（BLL）	③	public	bool fetchCartItems(QJsonArray &cartItems, QString &errorMessage);
数据访问层（DAL）	④	public	bool fetchCartItems(const QString &username, QJsonArray &cartItems, QString &errorMessage);

①当网络通信层收到拉取购物车信息操作时，handleFetchCartRequest 被调用，该方法构建一个字符串类型的错误信息和数据包信息，调用业务逻辑层的购物车类的方法 fetchCartItems

③业务逻辑层的 fetchCartItems 被调用，该函数继续调用数据访问层的购物车数据访问类的方法 fetchCartItems，将错误信息和数据包信息继续向下传递。

④数据访问层的 fetchCartItems 完成数据打包，填写错误信息和数据包信息。结束调用

③业务逻辑层的 fetchCartItems 结束调用，将错误信息和数据包信息继续向上传递。

①handleFetchCartRequest 读取错误信息和数据包信息，构造好应答报文，调用 sendJsonData 发送出去。

②sendJsonData 被调用，信息发送给客户端。

几乎服务端的所有请求均按照如上实现，构成服务端的逻辑核心。

3.3 数据库说明

系统用到的数据文件的格式说明；系统用到的关系数据库的表的说明（包括每个字段的名称，类型，说明等）

3.3.1 文件说明

应题目要求，用户的相关信息存储在文件中，数据结构如下。

```
/**
 * @brief 用户数据结构
 */
struct UserData {
    unsigned int id; // 用户 ID
    QString name; // 用户名
    QString password; // 密码
    unsigned long long balance; // 余额(分为单位)
    QString type; // 用户类型(Customer/Merchant)
    QString storeName; // 商店名称(商家专用)
```

```
    QMap<QString, float> uniDiscounts; // 统一折扣设置(商品类型->折扣值)
};
```

为了便于写入读出，这里使用 json 文件存储。文件中存储的用户信息基本如下：

```
{
    "nextUserId": 4,
    "users": [
        {
            "balance": 56627,
            "id": 1,
            "name": "1",
            "password": "1",
            "storeName": "",
            "type": "Customer",
            "uniDiscounts": {
                "Book": 1,
                "Clothing": 1,
                "Food": 1
            }
        },
        省略下文
```

其中 nextUserId 存储下一个用户的 id，每一个商家存储了一个统一折扣值。

3.3.2 关系数据库说明

下面使用 DDL (Data Definition Language) 辅以说明来展示数据库的结构：

3.3.2.1 商品数据库

```
CREATE TABLE `commodity_table` (
  `id` int unsigned NOT NULL AUTO_INCREMENT,
  `name` varchar(45) NOT NULL,
  `type` varchar(45) NOT NULL,
  `description` varchar(45) DEFAULT NULL,
  `originalPrice` bigint unsigned NOT NULL,
  `stock` int unsigned DEFAULT '0',
  `sourceStoreName` varchar(45) NOT NULL,
  `discount` float unsigned NOT NULL,
  `uniDiscount` float unsigned NOT NULL,
  PRIMARY KEY (`id`)
) ENGINE=InnoDB AUTO_INCREMENT=20 DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_0900_ai_ci
```

辅助说明：

字段名	说明
id	商品的唯一标识符，**主键**，自动递增。
name	商品的名称，非空。
type	商品的类型或分类，非空。
description	商品的详细描述，可为空。
originalPrice	商品的原始价格，非空。
stock	商品的当前库存量，默认为 0。
sourceStoreName	商品的来源店铺名称，非空。
discount	商品的折扣率，非空。
uniDiscount	商品的统一折扣率，非空。

3.3.2.2 订单数据库

```
CREATE TABLE `order_table` (
  `id` int unsigned NOT NULL AUTO_INCREMENT,
  `user` varchar(45) NOT NULL,
  `totalPrice` bigint unsigned NOT NULL,
  `time` varchar(25) NOT NULL,
  `state` varchar(45) NOT NULL DEFAULT 'unPaid',
  PRIMARY KEY (`id`),
  UNIQUE KEY `idorder_table_UNIQUE` (`id`),
  KEY `user_idx` (`user`)
) ENGINE=InnoDB AUTO_INCREMENT=79 DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_0900_ai_ci
```

辅助说明：

唯一键： idorder_table_UNIQUE (id) (与主键重复，是可省略的)

索引： user_idx (user) 用于加速按用户查询订单。

字段名	说明
id	订单的唯一标识符，**主键**，自动递增。
user	下单用户的标识符，非空。
totalPrice	订单的总价格，非空。
time	订单生成的时间戳，非空。
state	订单的当前状态，默认为“unPaid”，非空。

3.3.2.3 具体的订单商品数据库

```
CREATE TABLE `orderitem_table` (
```

```
  `id` int unsigned NOT NULL,
  `commodityId` int unsigned NOT NULL,
  `commodityName` varchar(45) DEFAULT NULL,
  `quantity` int unsigned DEFAULT NULL,
  `originalPrice` bigint unsigned DEFAULT NULL,
  `discount` float unsigned DEFAULT NULL,
  `uniDiscount` float unsigned DEFAULT NULL,
  PRIMARY KEY (`id`,`commodityId`),
  KEY `fk_commodityId_idx` (`commodityId`),
  KEY `fk_commodity_id_idx` (`commodityId`),
  CONSTRAINT `fk_commodity_id` FOREIGN KEY (`commodityId`) REFERENCES `commodity_table` (`id`) ON DELETE CASCADE ON UPDATE RESTRICT,
  CONSTRAINT `fk_id` FOREIGN KEY (`id`) REFERENCES `order_table` (`id`) ON DELETE CASCADE ON UPDATE RESTRICT
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_0900_ai_ci
```

辅助说明：

主键:id, commodityId (复合主键)

外键:fk_commodity_id (commodityId) 引用 commodity_table 的 id。

行为：ON DELETE CASCADE (删除商品时，订单项也删除)

行为：ON UPDATE RESTRICT (更新商品 ID 时，不允许)

fk_id (id) 引用 order_table 的 id。

行为：ON DELETE CASCADE (删除订单时，订单项也删除)

行为：ON UPDATE RESTRICT (更新订单 ID 时，不允许)

索引:fk_commodityId_idx (commodityId)加快查找速度。

字段名	说明
id	订单项所属订单的 ID，**复合主键**的一部分，**外键**引用 order_table。
commodityId	订单项对应的商品 ID，**复合主键**的一部分，**外键**引用 commodity_table。
commodityName	订单项中商品的名称，可为空。
quantity	订单项中商品的购买数量，可为空。
originalPrice	订单项中商品的原始价格，可为空。
discount	订单项中商品的折扣率，可为空。
uniDiscount	订单项中商品的统一折扣率，可为空。

3.3.2.4 购物车数据库

```
CREATE TABLE `shoppingcart_table` (
  `id` int unsigned NOT NULL AUTO_INCREMENT,
  `user` varchar(45) NOT NULL,
  `commodityId` int unsigned NOT NULL,
  `quantity` int unsigned NOT NULL,
```

```
  PRIMARY KEY (`id`,`user`,`commodityId`),
  KEY `commodityId_idx` (`commodityId`),
  CONSTRAINT `fk_commodityId` FOREIGN KEY (`commodityId`) REFERENCES `commodity_table` (`id`) ON DELETE CASCADE
) ENGINE=InnoDB AUTO_INCREMENT=131 DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_0900_ai_ci
```

辅助说明：

主键:id, user, commodityId (复合主键)

外键:fk_commodityId (commodityId) 引用 commodity_table 的 id。

行为：ON DELETE CASCADE (删除商品时，购物车项也删除)

索引:commodityId_idx (commodityId)

字段名	说明
id	购物车项的唯一标识符，**复合主键**的一部分，自动递增。
user	购物车所属用户的标识符，**复合主键**的一部分。
commodityId	购物车中商品的 ID，**复合主键**的一部分，**外键**引用 commodity_table。
quantity	购物车中商品的数量，非空。

3.4 接口协议说明

本系统子系统间采用自定义的应用层通信协议，基于 TCP/IP 协议栈进行承载。

3.4.1 协议数据单元 (PDU) 的语法

每个协议数据单元 (PDU) 包含两部分：一个固定长度的头部（用于指示消息体长度）和可变长度的消息体（JSON 数据）。

结构示意图：

```
+---------------------+--------------------+
| 消息体长度（4 字节）| 消息体（JSON 数据）|
+---------------------+--------------------+
```

字段描述：

消息体长度 (4 字节)：

数据类型： quint32 (无符号 32 位整数)。

编码方式： 大端字节序，通过 qToBigEndian() 转换，确保跨平台兼容性。

含义： 表示紧随其后的 JSON 消息体的字节长度。

消息体 (JSON 数据)：

数据类型： 字符串 (UTF-8 编码)。

编码方式： 标准 JSON 格式，采用紧凑模式 (QJsonDocument::Compact)，不包含额外的空白字符。

含义： 承载实际的业务数据和控制信息，以键值对的形式组织。

3.4.2 协议数据单元 (PDU) 的语义

协议数据单元的语义定义了各字段的业务含义和处理规则：
消息体长度字段的语义： 接收方首先读取这 4 个字节，以确定后续需要读取多少字节才能获得完整的 JSON 消息。这解决了 TCP 流式传输中“粘包/分包”问题，确保每个逻辑消息的完整性。
JSON 消息体的语义：JSON 对象中的每个键值对代表一个特定的数据项。例如，"mtype" 键通常用于定义消息的类型（如 "loginRequest"、"dataUpdate"等），接收方根据此类型进行相应的业务逻辑处理。其他键（如 "username", "password", "data" 等）的语义则依赖于 "mtype" 字段所定义的具体消息类型。
仅举一例，示例 JSON 消息体结构：

```
用户注册请求
{
    "type": "register",
    "username": "newuser",
    "password": "password",
    "user_type": "Consumer" // 或 "Merchant"
}
注册成功回复
{
    "type": "register_response",
    "status": "success",
    "message": "User registered successfully."
}
用户已存在回复
{
    "type": "register_response",
    "status": "error",
    "message": "Username already exists."
}
数据库错误回复
{
    "type": "register_response",
    "status": "error",
     "message": "Database error during registration."
}
```

3.4.3 底层承载协议

本自定义协议运行在 TCP/IP 之上，借助 Qt 的 QTCPServer 建立连接。

3.5 UI 说明

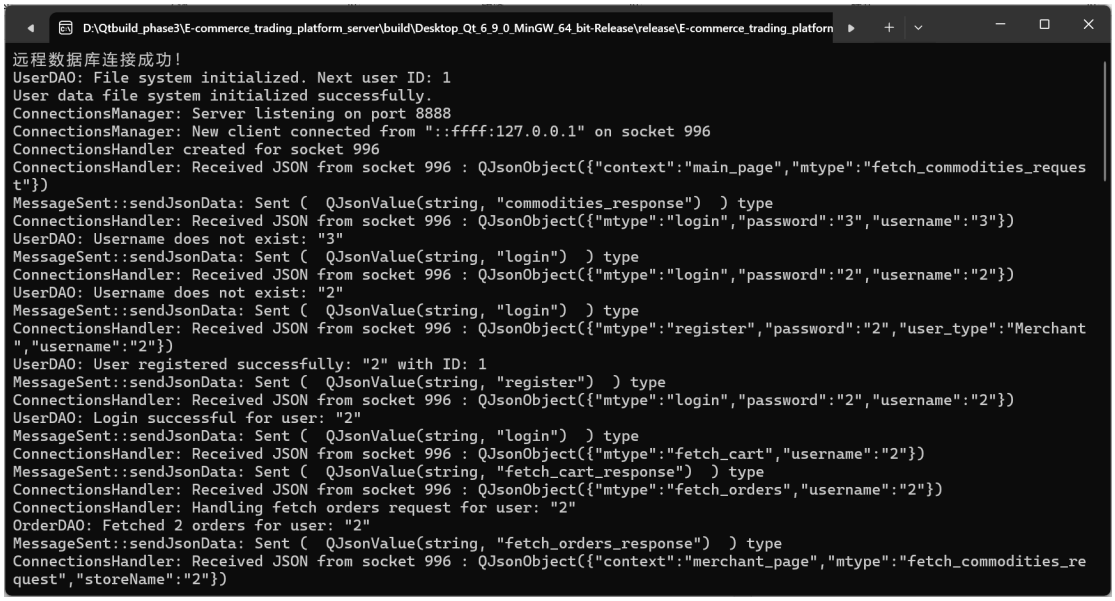
本系统使用 Qt Widget 进行 UI 的美化，主要在客户端的表示层 UIL 已经体现。即通过定义一个一个的 Qt widget 组件，并初始化布局，将组件加入布局，最后通过槽函数将用户操作和 UI 中的 widget 改变交互起来。同时使用了 qss 来美化界面。这部分与课程相关程度不大，不再具体说明。下面附几张图片展示 UI。

3.5.1 客户端展示

电商交易平台
你好， 2
余额：0.00
切换账号
注销
修改密码
充值
主页
购物车
我的订单
商品管理
商品名称
种类
单价
数量
总价
库存
操作
马原课本
Book
15.00
1
15.00
3
删除
葡萄
Food
8.00
1
8.00
66
删除
清空购物车
生成订单

电商交易平台
你好， 2
余额：0.00
切换账号
注销
修改密码
充值
主页
购物车
我的订单
商品管理
订单ID: 79
状态: unPaid
总价: 4.50
商品: 苹果
订单ID: 69
状态: paid
总价: 4.50
商品: 苹果
订单ID: 68
状态: paid
总价: 4.50
商品: 苹果
订单ID 79
用户名 2
总价 4.50
状态 unPaid
创建时间 2025-06-11 15:20:22
名称 原价 折后价 数量
苹果 5.00 4.50 1
总价: 4.50
删除订单
支付订单

3.5.2 服务端展示



```
D:\Qtbuild_phase3\E-commerce_trading_platform_server\build\Desktop_Qt_6_9_0_MinGW_64_bit-Release\release\E-commerce_trading_platforn
远程数据库连接成功！
UserDAO: File system initialized. Next user ID: 1
User data file system initialized successfully.
ConnectionsManager: Server listening on port 8888
ConnectionsManager: New client connected from "::ffff:127.0.0.1" on socket 996
ConnectionsHandler created for socket 996
ConnectionsHandler: Received JSON from socket 996 : QJsonObject({"context":"main_page","mtype":"fetch_commodities_reques
t"})
MessageSent::sendJsonData: Sent (  QJsonValue(string, "commodities_response")  ) type
ConnectionsHandler: Received JSON from socket 996 : QJsonObject({"mtype":"login","password":"3","username":"3"})
UserDAO: Username does not exist: "3"
MessageSent::sendJsonData: Sent (  QJsonValue(string, "login")  ) type
ConnectionsHandler: Received JSON from socket 996 : QJsonObject({"mtype":"login","password":"2","username":"2"})
UserDAO: Username does not exist: "2"
MessageSent::sendJsonData: Sent (  QJsonValue(string, "login")  ) type
ConnectionsHandler: Received JSON from socket 996 : QJsonObject({"mtype":"register","password":"2","user_type":"Merchant
","username":"2"})
UserDAO: User registered successfully: "2" with ID: 1
MessageSent::sendJsonData: Sent (  QJsonValue(string, "register")  ) type
ConnectionsHandler: Received JSON from socket 996 : QJsonObject({"mtype":"login","password":"2","username":"2"})
UserDAO: Login successful for user: "2"
MessageSent::sendJsonData: Sent (  QJsonValue(string, "login")  ) type
ConnectionsHandler: Received JSON from socket 996 : QJsonObject({"mtype":"fetch_cart","username":"2"})
MessageSent::sendJsonData: Sent (  QJsonValue(string, "fetch_cart_response")  ) type
ConnectionsHandler: Received JSON from socket 996 : QJsonObject({"mtype":"fetch_orders","username":"2"})
ConnectionsHandler: Handling fetch orders request for user: "2"
OrderDAO: Fetched 2 orders for user: "2"
MessageSent::sendJsonData: Sent (  QJsonValue(string, "fetch_orders_response")  ) type
ConnectionsHandler: Received JSON from socket 996 : QJsonObject({"context":"merchant_page","mtype":"fetch_commodities_re
quest","storeName":"2"})
```

服务端主要是打印数据，没有过分强调美观。

4 实现

4.1 单机版实现过程的问题：

单机版实现过程的问题主要是新环境的熟悉问题和面向对象的设计问题。

4.1.1 新环境的熟悉问题

1. 刚开始做实验时，我打算使用数据库，因为我不确定如何进行数据库的操作，同时又因为对于数据库了解不多，选用了 MySQL 数据库，该数据库可能较为复杂，需要复杂的配置（主要是 Qt6 以后不再提供现成的驱动，需要另行下载配置）和操作（需要对于 MySQL 的语法进行学习）。

   解决方法：好在距离期末较远，我仍旧有时间进行学习配置，花费了大量时间基本能够熟练使用 MySQL 和 Qt 的基本功能。

4.1.2 面向对象的整体设计问题

1. 整体设计没有区分开：Qt 的窗口类内容过多，用户类商品类过于单薄，本应在用户类和商品类中实现的功能竟然直接在窗口类中进行实现了。

   原因：既然打算制作 UI，我便在刚开始就使用 Qt 进行操作，防止后期改动巨大。然而由于我对 Qt Widget 的带界面开发流程不甚理解，且 Qt Widget 开发几乎与 C++的面向对象浑然天成（比如窗口也是类，组件也是类），导致我在整体设计上未能正确的做出分析，我不知道一些功能应该实现在窗口类里，还是实现在用户类里，是不是用户类和窗口类之间应该有什么关系？

   解决方法：这个问题并没有解决，因为我缺乏相关设计经验，只能慢慢尝试。网络版做了改进，但是这个问题仍然存在。单机版制作完毕后，我的用户类和商品类显得空虚，因为大部分功能直接在窗口类中进行实现了。

2. 数据库也同样影响我对类的理解，我想不明白的是：既然都有数据库了，为什么还要类的成员变量再存储一遍信息？

   解决方法：这个问题同样没有解决。单机版因为我不确定如何设计，所以我使用成员变量重新存储了一遍信息，但是改动时总是发生数据库和本地的内存状态不一致，甚至有时候本地的错误内容还会错误提交进而覆盖数据库。在网络版时有所思考，使得问题有所减轻。

4.1.3 其他具体问题

1. 我初始时采用 double 类型存储用户的余额，导致数据库存储常常不准确，后续第二个任务使用 unsigned long long 类型存储改正了这个问题。
2. 统一折扣开始理解错误，我误认为是整个平台的统一折扣。所以直接将统一折扣作为一个商品类共有的 static 变量存储。这样实现相当荒唐。后续在网络版得到了改正。
3. 一些方法未加 const，没有养成写代码的良好习惯，后续手动进行了大量的修改。

4.2 网络版实现过程的问题：

4.2.1 仍旧存在的面向对象的整体设计问题：

在网络版里，我刚开始做了一段时间的设计。我将窗口类和用户类的职责分离。窗口类只是负责做一系列窗口 UI 的变换（要负责每个控件什么时候隐藏，什么时候出现，每个表格的内容变换等），真正涉及到用户的行为这一抽象层面上的实现，则统一调用相关的类来处理。对于数据库和成员变量的问题，我思考过后认为在客户端常常需要一些直接从本地变量读取的操作，不应该每一次都去发送数据包查询数据库，所以成员变量在客户端是必定要存在的。而服务端对于成员变量的使用并不大，更多的感觉是一个抽象的类，重点在于方法。所以其实成员变量并没有多大用途，仅仅当作临时变量存储，所以仅仅在部分情况下，为了便捷，才维持了成员变量。
根据上述思考，最后得出了前文提到的分层模型。然而面向对象的整体设计问题仍然存在，主要表现在

1. 很多地方没有严格按照分层划分来做，有些地方比较繁琐（比如服务端的订单处理），受限于时间精力，可能没有按照分层，而是直接进行了数据库操作。这样一时简便，却为后续修改带来了困难。
2. 一些用户类的方法本应该放到商家类里面，却直接写在了用户类里，商家类里面是空的。这是因为之前的版本是这样实现的，保持原状可以不做过多的改动。但是这不是面向对象的设计，为后期维护带来了困难。

4.3 经验教训：

在本次项目实现过程中，从单机版到网络版的迭代开发，我积累了宝贵的经验，也深刻认识到了一些设计与实践上的教训。

1. 要时刻保持面向对象的设计观念。面向对象的一个核心是职责分离。比如明确区分界面层（UIL）与业务逻辑层（BLL）的职责。界面类应专注于用户交互和数据展示，而业务逻辑应封装在独立的、职责单一的类中（如用户类、商品类）。网络版中尝试将窗口类仅负责 UI 变换，而将用户行为抽象到独立类中，是迈向正确设计方向的一步。
2. 尽可能做好概要设计。在项目初期，由于缺乏经验或为了“一时简便”，将本应属于业务逻辑的功能直接实现在 UI 层（如单机版中窗口类内容过多），或者将方法放置在不恰当的类中（如网络版中用户类包含商家方法），会严重违反单一职责原则，导致代码臃

肿、耦合度高、难以理解、维护和扩展。这种实现在项目后期会成倍地增加修改成本和引入新错误的风险。虽然 C++大作业可能就此告一段落了，但是这样的设计习惯必须抛弃。

3. 持续练习，养成良好的代码习惯。即使在设计阶段提出了分层模型，但在实际编码过程中，由于时间或精力限制，未能严格按照设计执行，直接进行数据库操作；或者是总是不加 const 等，这表明将设计理念贯彻到每一行代码中需要持续的自律和经验积累。
4. 数据类型选择的严谨性。对于涉及精确计算的数值（尤其是金额），选择正确的数据类型至关重要。将金额存储为 unsigned long long（通过以“分”为单位）而非 double，有效避免了浮点数运算带来的精度问题，确保了数据准确性。
5. 迭代与反思的重要性。软件开发是一个不断迭代和完善的过程。从单机版到网络版的演进，每一次重构和改进都是对前期问题的一次修正和深化理解。通过不断地实践、发现问题、解决问题、再反思，能够逐步积累设计经验，提升开发能力。不要期望第一次就能做出完美的设计。关键在于勇于面对问题，积极寻找解决方案，并从中吸取教训。每一次遇到的困难都是一次学习的机会，促使我们更深入地理解面向对象原则、系统架构和最佳实践。